CUADERNO DE ENTRENAMIENTO SQL

Triggers y Procedimientos Almacenados para el Examen de Laboratorio (tiendaOnline)

Este cuaderno recopila de manera limpia, estructurada y explicada paso a paso todos los ejercicios prácticos de **Triggers** y **Procedures** que hemos trabajado para tu asignatura. Está optimizado para que puedas repasar la mecánica exacta exigida en los laboratorios de la Universidad de Sevilla.

BLOQUE 1: REGLAS DE NEGOCIO Y DISPARADORES (TRIGGERS)

Ejercicio T1: Validación básica en inserciones (BEFORE INSERT)

Enunciado: Crea un trigger llamado `trg_unidades_linea_pedido` que, *antes de insertar* una nueva línea en la tabla `LineasPedido`, compruebe si las `unidades` que se quieren comprar son mayores que 0 y menores o iguales que 100. Si no se cumple la condición, debe detener la base de datos lanzando un error mediante `SIGNAL SQLSTATE '45000'`.

Receta mecánica aprendida:

- Al validar datos antes de guardar, se usa siempre **BEFORE INSERT**.
- Los nuevos datos que intentan entrar viajan en la pseudotabla temporal **NEW**.
- No se debe hacer nunca un `SELECT ... FROM` sobre la misma tabla del trigger para evitar errores de mutación.

Solución Perfecta (100% Nota):

```
DELIMITER //

CREATE OR REPLACE TRIGGER trg_unidades_linea_pedido
BEFORE INSERT ON LineasPedido
FOR EACH ROW
BEGIN

    -- Evaluamos directamente el dato del futuro con NEW
    IF NOT (NEW.unidades > 0 AND NEW.unidades <= 100) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Las unidades deben encontrarse entre 1 y 100.';
    END IF;

END //

DELIMITER ;
```

Ejercicio T2: Validación cruzada de modificaciones (BEFORE UPDATE)

Enunciado: Crea un trigger llamado `trg_evitar_cambio_cliente` que salte en la tabla `Pedidos` cada vez que se intente modificar un registro. Si el usuario intenta cambiar el `clienteId` de ese pedido (asignarlo a otro cliente diferente), el trigger debe impedir el cambio lanzando una excepción.

Receta mecánica aprendida:

- En un **UPDATE** conviven el pasado (**OLD**) y el futuro (**NEW**) a la vez.
- Para saber si un campo ha cambiado, comparamos si el valor viejo es distinto al nuevo usando el operador **<>**.

Solución Perfecta (100% Nota):

```
DELIMITER //

CREATE OR REPLACE TRIGGER trg_evitar_cambio_cliente
BEFORE UPDATE ON Pedidos
FOR EACH ROW
BEGIN

    -- Comparamos el pasado con el futuro para cazar el cambio
    IF (NEW.clienteId <> OLD.clienteId) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No está permitido cambiar el cliente de un pedido ya
creado.';
    END IF;

END //

DELIMITER ;
```

Ejercicio T3: Modificación automática en tablas cruzadas (AFTER INSERT)

Enunciado: Crea un trigger llamado `trg_restar_stock_pedido` que salte en la tabla `LineasPedido`. Cada vez que se inserte una nueva línea de pedido, el trigger debe actualizar de forma automática la tabla `Productos` restando las unidades compradas del stock actual de dicho producto.

Receta mecánica aprendida:

- Como la validación ya pasó y queremos alterar otra tabla diferente, el momento correcto es **AFTER INSERT**.
- Atajo de examen: Se puede operar matemáticamente directo en la columna (`SET stock = stock - NEW.unidades`) sin declarar variables locales ni hacer un `SELECT INTO`.

Solución Perfecta (100% Nota):

```
DELIMITER //

CREATE OR REPLACE TRIGGER trg_restar_stock_pedido
AFTER INSERT ON LineasPedido
FOR EACH ROW
BEGIN

    -- Modificamos la estantería de Productos cruzando el ID
    UPDATE Productos
    SET stock = stock - NEW.unidades
    WHERE id = NEW.productoId;

END //

DELIMITER ;
```

BLOQUE 2: LOGICA TRANSACCIONAL (PROCEDURES)

Ejercicio P1: Transacción de inserción compleja con variables

Enunciado: Desarrolla un procedimiento almacenado y transaccional llamado `p_registrarLineaPedido` que sirva para añadir un producto a un pedido existente. Recibe el ID del pedido (`p_pedidoId`), el ID del producto (`p_productoId`) y las unidades (`p_unidades`). Debe buscar el precio actual del producto en la tabla `Productos`, guardarlo en una variable local, e insertar la nueva fila en `LineasPedido` con ese precio capturado.

Receta mecánica aprendida:

- Es obligatorio el "Airbag" de examen para evitar notas nulas: `DECLARE EXIT HANDLER FOR SQLEXCEPTION`.
- Las variables locales se declaran estrictamente arriba, justo bajo el primer `BEGIN`.
- Abrimos zona segura con **START TRANSACTION;** y cerramos con **COMMIT;**.

Solución Perfecta (100% Nota):

```
DELIMITER //

CREATE OR REPLACE PROCEDURE p_registrarLineaPedido(
    IN p_pedidoId INT,
    IN p_productoId INT,
    IN p_unidades INT
)
BEGIN
    -- 1. Declaración de variables locales de trabajo
    DECLARE v_precio DECIMAL(10,2);

    -- 2. Airbag de seguridad obligatorio para examen
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK; -- Deshace los cambios si hay error para no corromper la BD
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error al registrar la línea de
pedido.';
    END;

    -- 3. Apertura de la zona transaccional
    START TRANSACTION;

    -- Rescatamos el precio del producto y lo guardamos en la variable v_precio
    SELECT precio INTO v_precio
    FROM Productos
    WHERE id = p_productoId;

    -- Insertamos de forma segura usando los parámetros y la variable
    INSERT INTO LineasPedido(pedidoId, productoId, unidades, precio)
    VALUES (p_pedidoId, p_productoId, p_unidades, v_precio);

    -- Confirmamos de forma permanente en el disco duro
    COMMIT;

END //

DELIMITER ;
```

Ejercicio P2: Mantenimiento seguro y borrado relacional en cascada manual

Enunciado: Desarrolla un procedimiento llamado `p_eliminarClienteSeguro` que reciba el ID de un cliente (`p_clienteId`) y elimine de forma segura todo su rastro. Debido a las restricciones `RESTRICT` de las claves ajenas, el procedimiento debe borrar de manera ordenada: primero las líneas de pedido de sus pedidos, luego sus pedidos, y finalmente la ficha del cliente.

Receta mecánica aprendida:

- Para evitar el **Error 1093 de MariaDB** (modificar/borrar una tabla usando un sub-SELECT de ella misma), filtramos en el primer `DELETE` por la columna `pedidoId IN (SELECT id FROM Pedidos...)`.
- El orden relacional estricto es: de Hijos/Nietos a Padres (Líneas ➔ Pedidos ➔ Clientes).

Solución Perfecta (100% Nota):

```
DELIMITER //

CREATE OR REPLACE PROCEDURE p_eliminarClienteSeguro(
    IN p_clienteId INT
)
BEGIN

    -- Airbag de examen obligatorio
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error al intentar eliminar el
cliente de forma segura.';
    END;

    -- Iniciamos la zona segura transaccional
    START TRANSACTION;

    -- PASO 1: Borramos las líneas (nietas) de todos los pedidos que hizo este cliente
    DELETE FROM LineasPedido
    WHERE pedidoId IN (SELECT id FROM Pedidos WHERE clienteId = p_clienteId);

    -- PASO 2: Borramos los pedidos (padres) de este cliente
    DELETE FROM Pedidos
    WHERE clienteId = p_clienteId;

    -- PASO 3: Borramos finalmente la ficha del cliente de la tabla principal
    DELETE FROM Clientes
    WHERE id = p_clienteId;
```

```
    -- Si todo ha ido bien, guardamos definitivamente
    COMMIT;

END //

DELIMITER ;
```

Ejercicio P3: Cálculos de agregación y retorno de información

Enunciado: Desarrolla un procedimiento llamado `p_calcularImportePedido` que reciba el ID de un pedido (`p_pedidoId`). El procedimiento debe calcular el importe total del pedido (sumando el resultado de multiplicar las unidades por el precio de todas sus líneas en `LineasPedido`), almacenar el resultado en una variable local y, finalmente, mostrar dicho valor por pantalla con el alias `'Importe Total'`.

Receta mecánica aprendida:

- Para agrupar varias filas matemáticas en un único número final se usa la función de agregación **SUM(...)**.
- Para volcar el resultado de una función de agregación en una variable se usa **INTO variable**.

Solución Perfecta (100% Nota):

```
DELIMITER //

CREATE OR REPLACE PROCEDURE p_calcularImportePedido(
    IN p_pedidoId INT
)
BEGIN
    -- Declaramos la variable local para guardar el dinero calculado
    DECLARE v_total DECIMAL(10,2);

    -- Airbag de examen obligatorio
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error al calcular el importe del
pedido.';
    END;

    -- Iniciamos la zona segura transaccional
    START TRANSACTION;

    -- Calculamos el sumatorio total del pedido y lo guardamos en v_total
    SELECT SUM(unidades * precio) INTO v_total
    FROM LineasPedido
```

```
    WHERE pedidoId = p_pedidoId;

    -- Mostramos limpiamente el valor de la variable por pantalla con su alias
    SELECT v_total AS 'Importe Total';

    -- Cerramos la transacción de consulta de forma limpia
    COMMIT;

END //

DELIMITER ;
```